LAPORAN PRAKTIKUM PEKAN 4

STRUKTUR DATA

“QUEUE”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan Queue menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2025 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 27 April 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

BAB II PEMBAHASAN 2

2.1 Pengertian Queue 2

2.2 Jenis – jenis Queue 3

2.3 Keuntungan dan Keterbatasan 3

2.4 Implementasi Penggunaan Queue 4

2.4.1 QueueArray_2511533011 4

2.4.2 QueueArrayDriver_2511533011 6

2.4.3 QueueLinkedList_2511533011 7

2.4.4 IterasiQueue_2511533011 9

2.4.5 ReverseData_2511533011 11

BAB III PENUTUP 13

3.1 Kesimpulan 13

3.2 Saran 14

DAFTAR PUSTAKA 15

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pernahkah melihat puluhan atau bahkan ratusan orang mengantri membeli makanan yang sedang viral pada saat itu, mereka rela mengantri berjam – jam untuk memenuhi rasa penasarannya tanpa harus bertengkar untuk mendapatkannya. Kenapa bisa begitu? karena store tersebut menggunakan sistem "pertama datang pertama dapat".

Jika dalam dunia informatika, efisiensi manajemen data sering kali mengadopsi prinsip-prinsip yang terjadi di kehidupan nyata. Salah satu struktur data linear yang paling fundamental adalah **Queue** atau antrean. Berbeda dengan *Stack* yang menggunakan prinsip LIFO (*Last In First Out*), Queue beroperasi berdasarkan prinsip **FIFO (First In First Out)**.

Prinsip ini menegaskan bahwa elemen data yang pertama kali masuk ke dalam struktur akan menjadi yang pertama kali diproses atau dikeluarkan. Secara teknis, operasi ini melibatkan dua ujung utama: **Front** (ujung depan) untuk penghapusan data (*Dequeue*) dan **Rear/Tail** (ujung belakang) untuk penambahan data (*Enqueue*).

1.2 Tujuan Praktikum

1. Memahami konsep dasar dan karakteristik struktur data **Queue** sebagai struktur data linear.
2. Memahami prinsip kerja **FIFO (First In First Out)** dalam pengelolaan elemen data.
3. Mengenali komponen-komponen utama dalam Queue, seperti **Front** (Head) dan **Rear** (Tail).

BAB II
PEMBAHASAN

2.1 Pengertian Queue

Queue atau dalam Bahasa Indonesia yang berarti antrean adalah struktur data yang menyusus elemen – elemen data dalam urutan linear. Prinsip dasar dari struktur data ini adalah ***"First In First Out"*** yang berartu elemen data yang pertama dimasukkan ke dalam antrean akan menjadi yang pertama pula untuk dikeluarkan.

1. Prinsip FIFO pada Queue

Caranya bekerja adalah seperti jejeran orang yang sedang menunggu antrean di supermarket di mana orang pertama yang datang adalah yang pertama dilayani (**First In, First Out**). Pada struktur data ini, urutan pertama (data yang akan dikeluarkan) disebut **Front** atau **Head**. Sebaliknya, data pada urutan terakhir (data yang baru saja ditambahkan) disebut **Back**, **Rear**, atau **Tail**. Proses untuk menambahkan data pada antrean disebut dengan **Enqueue**, sedangkan proses untuk menghapus data dari antrean disebut dengan **Dequeue**.

2. Fungsi Queue

Queue memiliki peran yang penting dalam berbagai aplikasi dan algoritma. Salah satu fungsi utamanya adalah mengatur dan mengelola antrean tugas atau operasi secara efisien. Dalam sistem komputasi, ia digunakan untuk menangani tugas-tugas seperti penjadwalan proses, antrean pesan, dan manajemen sumber daya.

2.2 Jenis – jenis Queue

A. Berdasarkan Implementasinya

1. Linear/Simple Queue

   Elemen – elemen data disusun dalam barisan linear dan penambahan serta penghapusan elemen hanya terjadi pada dua ujung barisan tersebut.

2. Circular Queue

   Mirip dengan jenis linear, tetapi ujung-ujung barisan terhubung satu sama lain, menciptakan struktur antrean yang berputar.

B. Berdasarkan Penggunaan

3. Priority Queue

   Setiap elemen memiliki prioritas tertentu. Elemen dengan prioritas tertinggi akan diambil terlebih dahulu.

4. Double-ended Queue (Dequeue)

   Elemen dapat ditambahkan atau dihapus dari kedua ujung antrean.

2.3 Keuntungan dan Keterbatasan

A. Keuntungan

- Data berjumlah besar dapat dikelola dengan mudah dan efisien.
- Proses *insert* dan *delete* data dapat dilakukan dengan mudah karena mengikuti aturan *first in first out* (FIFO).
- Efisien dalam menangani tugas berdasarkan ururtan kedatangan.

B. Keterbatasan

- Tidak efisien untuk penvarian elemen tertentu dalam antrean.
- Memerlukan alokasi memori yang cukup untuk menyimpan antrean.

2.4 Implementasi Penggunaan Queue

2.4.1 QueueArray_2511533011

```java
package pekan4_2511533011;

public class QueueArray_2511533011 {
int front_3011, rear_3011, size_3011;
int capasity_3011;
int array_3011[];

public QueueArray_2511533011(int capasity) {
    this.capasity_3011 = capasity;
    front_3011 = this.size_3011 = 0;
    rear_3011 = capasity -1;
    array_3011 = new int[this.capasity_3011];
}

boolean isFull_2511533011(QueueArray_2511533011 queue) {
    return (queue.size_3011 == queue.capasity_3011);
}

boolean isEmpty_2511533011(QueueArray_2511533011 queue) {
    return (queue.size_3011 == 0);
}

void enqueue_2511533011 (int item) {
    if (isFull_2511533011(this))
        return;
    this.rear_3011 = (this.rear_3011 + 1) % this.capasity_3011;
    this.array_3011[this.rear_3011] = item;
    this.size_3011 = this.size_3011 + 1;
    System.out.println(item + " enqueued to queue ");
}
 int dequeue_3011() {
     if(isEmpty_2511533011(this))
         return Integer.MIN_VALUE;
     int item = this.array_3011[this.front_3011];
     this.front_3011 = (this.front_3011 + 1) % this.capasity_3011;
     this.size_3011 = this.size_3011 -1;
     return item;
 }

 int front_3011() {
     if (isEmpty_2511533011 (this))
         return Integer.MIN_VALUE;
```

Gambar 2.6.1

```java
        return this.array_3011[this.rear_3011];
    }

    int rear_3011 () {
            if (isEmpty_2511533011(this))
                return Integer.MIN_VALUE;
            return this.array_3011[this.rear_3011];
    }

        //mencetak elemen antrian
    void display_3011() {
        int i;
        if (front_3011 == rear_3011) {
            System.out.println("\nAntrian Kosong\n");
            return;
        }

        //kunjungi dari belakang dan cetak
        for (i = front_3011; i < rear_3011; i++) {
            System.out.printf(" %d <-- ", array_3011[i]);
        }
        return;
    }
}
```

Gambar 2.6.1

Class ini merupakan implementasi **Circular Queue** menggunakan array. Penggunaan operator modulo (%) menunjukkan bahwa antrean ini memanfaatkan ruang array secara melingkar agar memori lebih efisien.

Atribut Utama :

- front_3011: Menyimpan indeks elemen terdepan yang akan diambil (keluar).
- rear_3011: Menyimpan indeks elemen terakhir yang baru saja dimasukkan.
- size_3011: Melacak jumlah elemen yang saat ini ada dalam antrean.
- capasity_3011: Menentukan batas maksimal data yang bisa ditampung.

Metode Utama :

- **Constructor**: Menginisialisasi kapasitas, mengatur size dan front ke 0, serta mengatur rear ke posisi akhir (capacity - 1) untuk mempersiapkan pengisian pertama.

- **isFull & isEmpty**: Fungsi pengecekan kondisi antrean berdasarkan nilai size_3011.
- **enqueue_2511533011**: Menambahkan data ke posisi belakang. Logika (rear + 1) % capacity memastikan jika rear mencapai batas akhir array, ia akan kembali ke indeks 0 jika masih ada ruang.
- **dequeue_3011**: Mengambil data dari posisi depan. Menggunakan logika (front + 1) % capacity untuk menggeser indeks depan secara sirkular.
- **display_3011**: Mencetak isi antrean dari indeks depan hingga belakang.

2.4.2 QueueArrayDriver_2511533011

```java
package pekan4_2511533011;

public class QueueArrayDriver_2511533011 {
    public static void main(String[] args) {
        QueueArray_2511533011 queue_3011 = new QueueArray_2511533011(1000);
        queue_3011.enqueue_2511533011(10);
        queue_3011.enqueue_2511533011(20);
        queue_3011.enqueue_2511533011(30);
        queue_3011.enqueue_2511533011(40);
        System.out.println("item di depan" + queue_3011.front_3011());
        System.out.println("item paling belakang " + queue_3011.rear_3011);
        System.out.println("tampilan queue");
        queue_3011.display_3011();
        System.out.println();
        System.out.println(queue_3011.dequeue_3011() + " dihapus dari queue ");
        System.out.println("item di depan: " + queue_3011.front_3011());
        System.out.println("item di belakang: " + queue_3011.rear_3011());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_3011.display_3011();
    }
}
```

Gambar 2.6.2

Class ini berfungsi sebagai **Main Class** untuk mendemonstrasikan cara kerja antrean yang telah dibuat.

Alur Eksekusi:

1. **Instansiasi**: Membuat objek queue_3011 dengan kapasitas 1000.
2. **Operasi Enqueue**: Memasukkan empat data (10, 20, 30, 40) ke dalam antrean. Berdasarkan prinsip FIFO, 10 berada di depan dan 40 di belakang.
3. **Inspeksi Data**: Mencetak elemen terdepan dan terbelakang menggunakan metode front_3011() dan rear_3011().
4. **Operasi Dequeue**: Menghapus satu elemen. Data yang terhapus adalah **10** (karena ia yang pertama masuk).
5. **Display**: Menampilkan sisa antrean (20, 30, 40) untuk membuktikan bahwa data telah tergeser secara logis.

2.4.3 QueueLinkedList_2511533011

```java
package pekan4_2511533011;
import java.util.LinkedList;

public class QueueLinkedList_2511533011 {
    public static void main(String[] args) {
        Queue<Integer> q_3011 = new LinkedList<>();
        // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i = 0; i < 6; i++)
            q_3011.add(i);
        // Menampilkan isi antrian
        System.out.println("Elemen Antrian" + q_3011);
        // Untuk menghapus kepala antrian.
        int hapus_3011 = q_3011.remove();
        System.out.println("Hapus elemen = " + hapus_3011);
        System.out.println(q_3011);
        // Untuk melihat antrian terdepan
        int depan_3011 = q_3011.peek();
        System.out.println("Kepala Antrian = " + depan_3011);

        int banyak_3011 = q_3011.size();
        System.out.println("size antrian = " + banyak_3011);
    }
}
```

Gambar 2.6.3

Class ini menunjukkan implementasi Queue yang lebih modern dan dinamis di Java. Pada class ini kita tidak perlu menentukan kapasitas maksimal di awal karena menggunakan LinkedList yang bersifat *dynamic resizing.*

Metode yang digunakan :

- **add(i)**: Menambahkan elemen ke akhir antrean (*enqueue*). Jika antrean penuh (pada kapasitas terbatas), ini akan melempar *exception*.
- **remove()**: Menghapus dan mengambil elemen terdepan (*dequeue*). Jika antrean kosong, metode ini akan melempar *exception*.
- **peek()**: Mengambil atau melihat elemen terdepan tanpa menghapusnya. Ini sangat berguna untuk pengecekan data sebelum diproses.
- **size()**: Mengembalikan jumlah elemen yang ada di dalam antrean saat ini.

Kenggulan :

1. Memori Dinamis : **isFull** (antrean penuh) kecuali memori computer benar – benar habis, karena LinkedList akan terus bertambah ukurannya sesuai kebutuhan.
2. Keamanan Tipe (Generics) : Penggunaan **<Integer>** memastikan bahwa antrean hanya bisa diisi oleh angka bulat, mencegah kesalahan tipe data saat dieksekusi.
3. Kemudahan Penulisan : Bisa langsung mencetak objek **q_3011** ke layer (System.out.println) dan Java akan otomatis menampilkan seluruh isi antrean berkat metode **toString()** yang sudah terintegrasi.

2.4.4 IterasiQueue_2511533011

```java
package pekan4_2511533011;
import java.util.Iterator;

public class IterasiQueue_2511533011 {
    public static void main(String[] args) {
        {
            Queue<String> q_3011 = new LinkedList<>();

            q_3011.add("Praktikum");
            q_3011.add("Struktur");
            q_3011.add("Data");
            q_3011.add("Dan");
            q_3011.add("Algoritma");
            Iterator<String> iterator_3011 = q_3011.iterator();
            while (iterator_3011.hasNext()) {
                System.out.print(iterator_3011.next() + " ");
            }
        }
    }
}
```

Gambar 2.6.4

Pada class ini, penggunaan **Iterator** untuk mengakses data dalam koleksi. Ini adalah cara standar dalam Java untuk membaca elemen dari awal hingga akhir secara berurutan.

1. Konsep Iterator pada Queue

   Jika ingin melihat isi Queue satu per satu, kita bisa menggunakan perulangan. Namun, **Iterator** memberikan cara yang lebih formal dan aman.

   - **q_3011.iterator()**: Metode ini membuat objek iterator yang "menunjuk" ke posisi sebelum elemen pertama dalam antrean.
   - **iterator_3011.hasNext()**: Sebuah fungsi pengecekan (boolean). Ia akan bernilai true jika masih ada elemen selanjutnya dalam antrean yang belum dikunjungi.

- **iterator_3011.next()**: Metode ini melakukan dua hal sekaligus: mengambil nilai elemen saat ini dan menggeser penunjuk (pointer) ke elemen berikutnya.

2. Perbedaan dengan Operasi Queue standar

   Melakukan iterasi tidak sama dengan melakukan **remove()** :

   - Iterasi hanya "berkunjung" dan membaca data. Setelah *loop* selesai, isi antrean **q_3011** tetap utuh dan tidak berubah.
   - **Remove/Dequeue** membaca data sekaligus menghapus dari struktur.

3. Implementasi String

   Berbeda dengan class sebelumnya yang menggunakan **Integer,** kali ini menggunakan tipe data **String.** Ini membuktikan bahwa interface **Queue<E>** di Java adalah Generic, yang artinya dapat menampung objek tipe apa pun. Contohnya seperti output pada class ini "Praktikum Struktur Data dan Algoritma".

2.4.5 ReverseData_2511533011

```java
package pekan4_2511533011;
import java.util.LinkedList;

public class ReverseData_2511533011 {
    public static void main(String[] args) {
        Queue<Integer> q_3011 = new LinkedList<Integer>();
        q_3011.add(1);
        q_3011.add(2);
        q_3011.add(3);  // [1, 2, 3]
        System.out.println("sebelum reverse = " + q_3011);
        Stack<Integer> s_3011 = new Stack<Integer>();
        while (!q_3011.isEmpty()) {
            s_3011.push(q_3011.remove());
        }
        while (!s_3011.isEmpty()) {
            q_3011.add(s_3011.pop());
        }
        System.out.println("sesudah reverse = " + q_3011); //   [3, 2, 1]

    }
}
```

Gambar 2.6.5

Class ini menggambarkan algoritma untuk membalikkan urutan elemen dalam sebuah **Queue** dengan bantuan **Stack**. Ini adalah contoh sederhana untuk menunjukkan perbedaan sifat dasar antara FIFO (*First In First Out)* dan LIFO *(Last In Last Out).*

Proses pembalikan data ini dilakukan dalam dua Langkah sistematis :

- Pemindahan ke Stack (Queue → Stack)

  Seluruh elemen dalam Queue diambil satu per satu menggunakan remove() dan dimasukkan ke dalam Stack menggunakan push(). Karena Queue bersifat FIFO, angka 1 keluar pertama, diikuti 2, lalu 3. Di dalam Stack, angka-angka ini menumpuk sehingga 3 berada di posisi paling atas.

- Pengembalian ke Queue (Stack → Queue)

  Elemen diambil dari Stack menggunakan pop() dan dimasukkan kembali ke Queue menggunakan add(). Karena Stack bersifat LIFO,

angka yang terakhir masuk (3) akan keluar pertama kali dan masuk ke antrean paling depan.

Analisis :

- Queue (FIFO) menyimpan data asli dan menampung hasil akhir.
- Stack (LIFO) sebagai media “pembalik” karena urutan keluar datanya berlawanan dengan urutan masuk.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa struktur data **Queue** merupakan fondasi penting dalam pemrograman yang menerapkan prinsip **First In First Out (FIFO)**. Melalui berbagai skenario pengujian, terlihat bahwa Queue dapat diimplementasikan secara manual melalui array dengan pendekatan **Circular Queue** yang efektif dalam mengoptimalkan penggunaan memori, maupun secara dinamis menggunakan **LinkedList** dalam Java Collections Framework. Implementasi manual memberikan pemahaman mendalam mengenai manipulasi penunjuk (*front* dan *rear*) serta pemanfaatan operator modulo untuk mencegah penumpukan indeks. Di sisi lain, penggunaan *library* bawaan menawarkan fleksibilitas tinggi, terutama dalam hal manajemen memori otomatis dan kemudahan akses data melalui **Iterator** yang memungkinkan pemeriksaan elemen tanpa merusak struktur antrean. Selain itu, praktikum ini membuktikan bahwa keterkaitan antar struktur data, seperti penggunaan **Stack** untuk membalikkan urutan Queue, menunjukkan bahwa pemahaman karakteristik dasar setiap struktur data sangat krusial dalam menentukan solusi algoritma yang efisien. Secara keseluruhan, penguasaan materi Queue ini memberikan dasar yang kuat bagi mahasiswa informatika dalam membangun sistem manajemen data yang terorganisir dan adaptif terhadap kebutuhan komputasi modern.

3.2 Saran

Untuk praktikum selanjutnya, disarankan agar materi dikembangkan dengan menambahkan studi kasus yang lebih dekat dengan kehidupan nyata, seperti simulasi sistem antrean di rumah sakit atau pengaturan lalu lintas data pada jaringan komputer. Selain itu, akan sangat menarik jika praktikum berikutnya mulai mengeksplorasi penggunaan **Priority Queue**, sehingga mahasiswa tidak hanya belajar antrean berdasarkan urutan waktu, tetapi juga berdasarkan tingkat kepentingan data. Penggunaan alat visualisasi tambahan atau pembuatan tampilan antrean yang lebih interaktif juga sangat disarankan agar pergerakan indeks *front* dan *rear* bisa lebih mudah dipahami secara visual, tidak hanya melalui angka-angka di terminal. Terakhir, analisis mengenai efisiensi waktu (kompleksitas algoritma) antara penggunaan array dan linked list perlu diperdalam agar mahasiswa semakin terbiasa membuat program yang tidak hanya benar secara logika, tetapi juga cepat dan efisien.

DAFTAR PUSTAKA

[1] R. Maulana, "Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya," dicoding, 29 November 2023. [Online]. Available: https://www.dicoding.com/blog/struktur-data-queue-pengertian-fungsi-dan-jenisnya/. [Accessed 28 April 2026].

TUGAS QUEUE

Soal

Mahasiswa diminta untuk mengimplementasikan struktur data Queue Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO First In First Out).

Program harus mampu:

- Menambahkan pelanggan ke antrian (enqueue)
- Menghapus pelanggan dari antrian (dequeue)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

Jawaban

1. Class Antrianloket_2511533011

```java
package pekan4_2511533011;

public class AntrianLoket_2511533011 {
int front_3011, rear_3011, max_3011 ;
String queue_3011[];

public AntrianLoket_2511533011(int max_3011) {
    this.max_3011 = max_3011;
    front_3011 = - 1;
    rear_3011 = - 1;
    queue_3011 = new String[this.max_3011];
}

boolean isFull_3011(AntrianLoket_2511533011 queue) {
    return (queue.rear_3011 == queue.max_3011 - 1);
}

boolean isEmpty_3011(AntrianLoket_2511533011 queue) {
    return (queue.front_3011 == - 1);
}

void enqueue_3011 (String people_3011) {
    if (isFull_3011(this))
        return;
    if (this.front_3011 == -1)
        this.front_3011 = 0;
    this.rear_3011 = (this.rear_3011 + 1) % this.max_3011;
    this.queue_3011[this.rear_3011] = people_3011;
    System.out.println("Data berhasil ditambahkan ke antrian ");
}
    String dequeue_3011() {
    if(isEmpty_3011(this))
        return null;
    String people_3011 = this.queue_3011[this.front_3011];
     if (this.front_3011 == this.rear_3011) {
            this.front_3011 = -1;
            this.rear_3011 = -1;
        } else {
            this.front_3011 = (this.front_3011 + 1) % this.max_3011;
        }
    return people_3011;
}
```

```java

    String front_3011() {
    if (isEmpty_3011 (this))
        return null;
    return this.queue_3011[this.rear_3011];
}

    //Mencetak elemen antrian
void display_3011() {
    int number_3011 = 1;
    if (isEmpty_3011(this)) {
        System.out.println("Antrian Kosong");
        return;
    }

    //Kunjungi dari belakang dan cetak
    for (int i_3011 = front_3011; i_3011 <= rear_3011; i_3011++) {
        System.out.println(number_3011 + ". " + queue_3011[i_3011]);
    }
    System.out.println();
}

void reverse_3011() {
    String[] temp = new String[max_3011];
    int count = 0;

    while (!isEmpty_3011(this)) {
        temp[count++] = dequeue_3011();
    }

    for (int i = count - 1; i >= 0; i--) {
        enqueue_3011(temp[i]);
    }
}
}
```

2. Class Queue_2511533011

```java
package pekan4_2511533011;
import java.util.Scanner;

public class Queue_2511533011 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLoket_2511533011 queue_3011 = new AntrianLoket_2511533011(20);

        int pilihan_3011;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_3011 = sc.nextInt();
            sc.nextLine(); // membersihkan buffer

            switch (pilihan_3011) {
                case 1:
                    if (queue_3011.isFull_3011(queue_3011)) {
                        System.out.println("Antrian sudah penuh!");
                    } else {
                        System.out.print("Masukkan nama pelanggan : ");
                        String nama = sc.nextLine();
                        queue_3011.enqueue_3011(nama);
                    }
                    break;

                case 2:
                    String keluar = queue_3011.dequeue_3011();
                    if (keluar == null) {
                        System.out.println("Antrian kosong, tidak ada yang dihapus!");
                    } else {
                        System.out.println(keluar + " telah dilayani");
                    }
                    break;

                case 3:
```

```java
                    System.out.println("\nIsi Antrian : ");
                    queue_3011.display_3011();
                    break;

                case 4:
                    if (queue_3011.isEmpty_3011(queue_3011)) {
                        System.out.println("Antrian kosong, tidak bisa di-reverse!");
                    } else {
                        queue_3011.reverse_3011();
                        System.out.println("Antrian berhasil di-reverse!");
                        queue_3011.display_3011();
                    }
                    break;

                case 5:
                    System.out.println("Terima kasih, program selesai!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
                    break;
            }

        } while (pilihan_3011 != 5);

        sc.close();
    }
}
```

Output

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan : Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan : Budi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

Isi Antrian :
1. Andi
1. Budi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
Antrian berhasil di-reverse!
1. Budi
1. Andi


=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

Isi Antrian :
1. Andi
```

Penjelasan singkat :

Kedua class ini membentuk satu kesatuan sistem simulasi antrian loket berbasis **Circular Queue** menggunakan array, di mana class AntrianLoket_2511533011 berperan sebagai penyedia logika struktur data seperti penambahan data (**enqueue**), penghapusan data (**dequeue**) dengan prinsip *First In First Out* (FIFO), pengecekan kondisi kosong atau penuh, hingga fungsi **reverse** untuk membalikkan urutan antrian. Sementara itu, class Queue_2511533011 bertindak sebagai program utama (*main class*) yang mengelola antarmuka interaktif bagi pengguna melalui menu pilihan yang dijalankan di dalam perulangan do-while dan percabangan switch-case untuk mengeksekusi fungsi-fungsi antrian tersebut secara langsung melalui input terminal.